Applications of Stacks

Dr. Priyanka N

Assistant Professor Senior Grade 1

School of Computer Science & Engineering

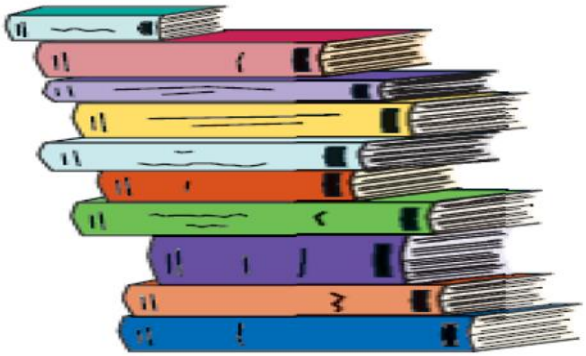
VIT, Vellore.

Module:1	Algorithm Analysis	8 hours
Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth - Algorithm efficiency – best case, worst case, average case - Analysis of non-recursive and recursive algorithms - Asymptotic analysis for recurrence relation: Iteration Method, Substitution Method, Master Method and Recursive Tree Method.		
Module:2	**Linear Data Structures**	**7 hours**
Arrays: 1D and 2D array- Stack - Applications of stack: Expression Evaluation, Conversion of Infix to postfix and prefix expression, Tower of Hanoi – Queue - Types of Queue: Circular Queue, Double Ended Queue (deQueue) - Applications – List: Singly linked lists, Doubly linked lists, Circular linked lists- Applications: Polynomial Manipulation.		
Module:3	**Searching and Sorting**	**7 hours**
Searching: Linear Search and binary search – Applications. Sorting: Insertion sort, Selection sort, Bubble sort, Counting sort, Quick sort, Merge sort - Analysis of sorting algorithms.		
Module:4	**Trees**	**6 hours**
Introduction - Binary Tree: Definition and Properties - Tree Traversals- Expression Trees:- Binary Search Trees - Operations in BST: insertion, deletion, finding min and max, finding the k^{th} minimum element.		
Module:5	**Graphs**	**6 hours**
Terminology – Representation of Graph – Graph Traversal: Breadth First Search (BFS), Depth First Search (DFS) - Minimum Spanning Tree: Prim's, Kruskal's - Single Source Shortest Path: Dijkstra's Algorithm.		
Module:6	**Hashing**	**4 hours**
Hash functions - Separate chaining - Open hashing: Linear probing, Quadratic probing, Double hashing - Closed hashing - Random probing – Rehashing - Extendible hashing.		
Module:7	**Heaps and AVL Trees**	**5 hours**
Heaps - Heap sort- Applications -Priority Queue using Heaps. AVL trees: Terminology, basic operations (rotation, insertion and deletion).		

Stack(ADT)

- What is Stack?
  - A stack is an ordered collection of homogeneous data element where the insertion and deletion take place at **one end**.
  - It is called a Last-in-First-Out (LIFO)collection.
  - It means, the item last entered will be first out.

Examples



Stack of Books

Plates in a tray

Goods in a cargo

Stacks Operations

- **What can we do with a stack?**
  - push – operation to place an item on the stack
  - pop – operation to look at the item on top of the stack and remove it
  - Traversal- Displaying the items on the stack
  - is Full - To know whether the stack is full or not
  - is Empty - To know whether the stack is empty or not
  -

Stack Applications

- It is a very common data structure
- Stacks are used in conversion of infix to postfix expression.
- Stacks are also used in evaluation of postfix expression.
- Stacks are used to implement recursive procedures (eg: Towers of Hanoi).
- Stacks are used in compiler design.
- Memory Management in Operating System.

- **Example 1: Balancing Symbols**
  - Make an empty Stack
  - Read characters until end of file
  - If the character is an opening symbol, push it onto the stack.
  - If it is a closing symbol and the stack is empty, report an error, Otherwise pop the element in stack.
  - If the symbol popped is not mapped with the corresponding opening symbol, then report an error.
  - At end of file, if the stack is not empty, report an error.

Example 2: Postfix Expressions

- Precedence of Operators - **specifies how "tightly" it binds two expressions together.**
- **For Example: 1+5*3= ??**
  - **16 or 18 ?? - Ans: 16 because** Multiplication ("*") operator has a higher precedence than the addition ("+") operator.

Typical Example:

- **4.99 * 1.06 + 5.99 + 6.99 * 1.06 = ???**
  - Multiply 4.99 and 1.06, saving this answer as **A_1**
  - Add 5.99 and A_1 and saves the result to **A_1**
  - Multiply 6.99 and 1.06, saving the answer in **A_2**
  - Finish by adding **A_1** and **A_2**, leaving the final answer in **A_1**

  - **Sequence of Operation is written as**

    4.99 1.06 * 5.99 + 6.99 1.06 * +

    - Post fix / Reverse Polish Notation

Post fix / Reverse Polish Notation

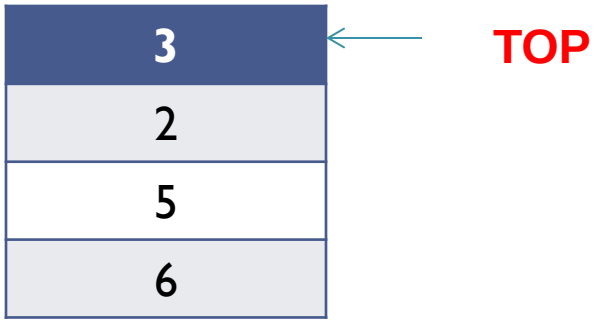
- **Evaluated easily by using stack**
  - When a number is seen, it is pushed onto the stack
  - When an operator is seen, the operator is applied to the two numbers (symbols) that are popped from the stack
  - Do the Operation and the result is pushed into the stack

Post fix / Reverse Polish Notation

- For instance, the postfix expression

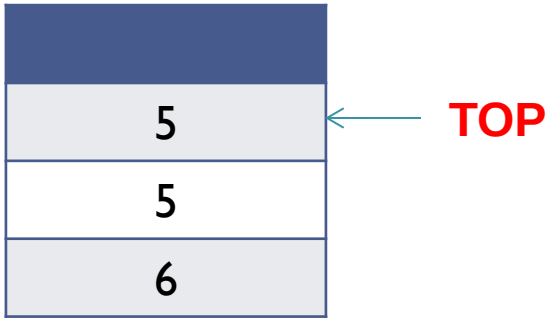
6 5 2 3 + 8 * +3 + *

- **First four symbols are placed on the stack**

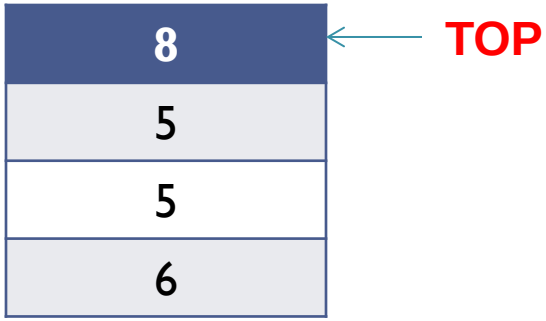


Post fix / Reverse Polish Notation

- Next '+' is read, so 3 and 2 are popped from the stack and their sum, 5, is pushed.

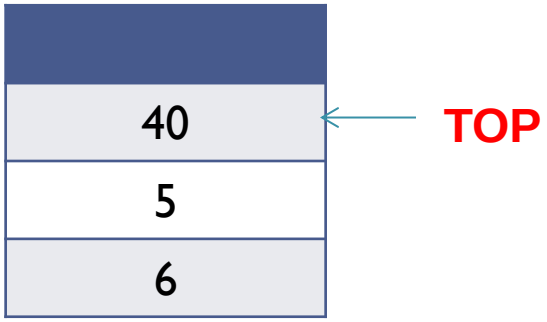


- Next '8' is read, so it is pushed into the stack.

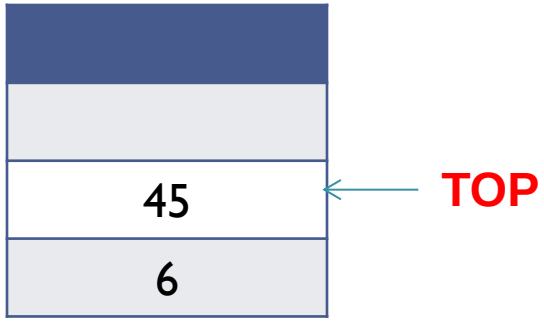


Post fix / Reverse Polish Notation

- Next '*' is read, so 8 and 5 are popped from the stack and their Product, 40, is pushed.

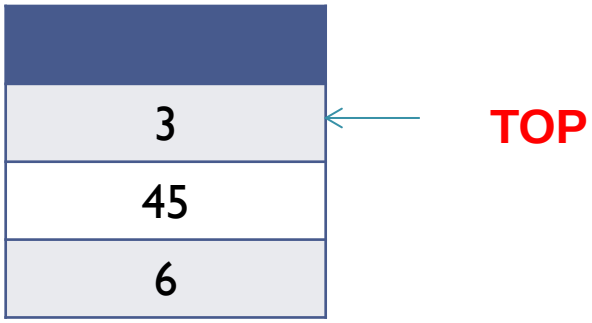


- Next '+' is read, so 40 and 5 are popped from the stack and their sum, 45, is pushed

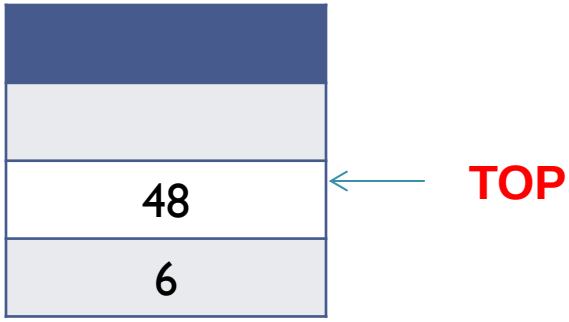


Post fix / Reverse Polish Notation

- Next '3' is read, so it is pushed into the stack.

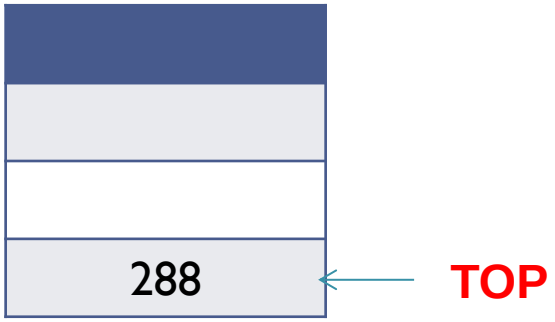


- Next '+' is read, so 45 and 3 are popped from the stack and their sum, 48, is pushed



Post fix / Reverse Polish Notation

- Next '*' is read, so 48 and 6 are popped from the stack and their Product, 288, is pushed.



- **Time to evaluate a postfix expression is O(N),** because processing each element in the input consists of stack operations and therefore takes constant time.

Example 3: Conversion of Infix to Postfix and Prefix Expression

- Infix to Postfix Conversion
  - Stack is used to convert an expression in standard form (otherwise known as infix) into postfix.
  - Example: **a + b * c + (d * e + f) * g**

Postfix Expression: a b c * + d e * f + g * +

Rules

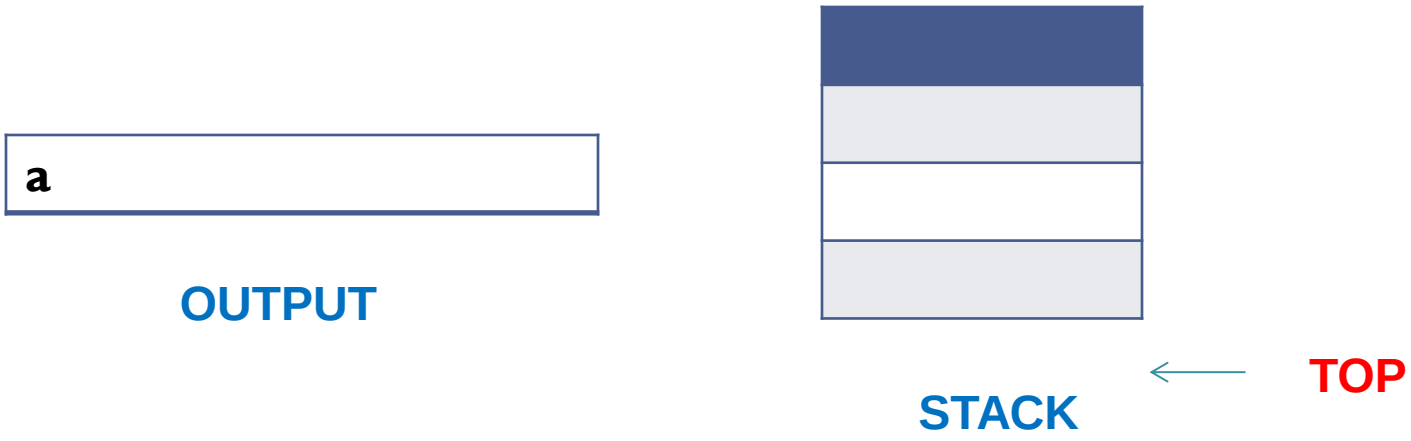
- When an operand is read, it is immediately placed onto the output.
- When an operator is seen, it is placed on the stack, stack represents pending operators.
- High precedence operator if present should be popped out before placing the current operator.(Except Parenthesis)

Infix to Postfix Conversion

- For instance, the postfix expression

  a + b * c + (d * e + f) * g

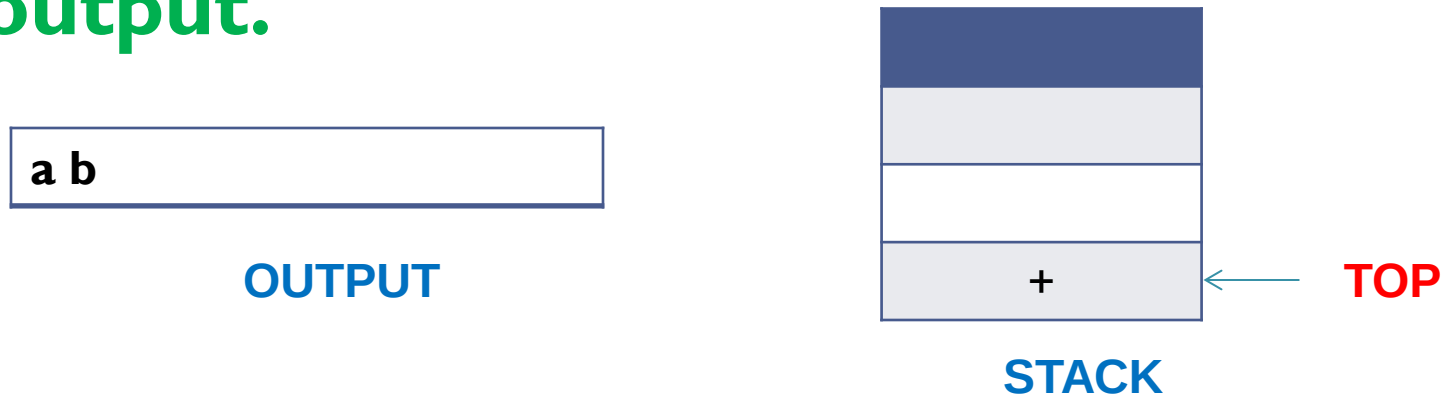
- **First the symbol "a" is read, it passed through the output.**



Infix to Postfix Conversion

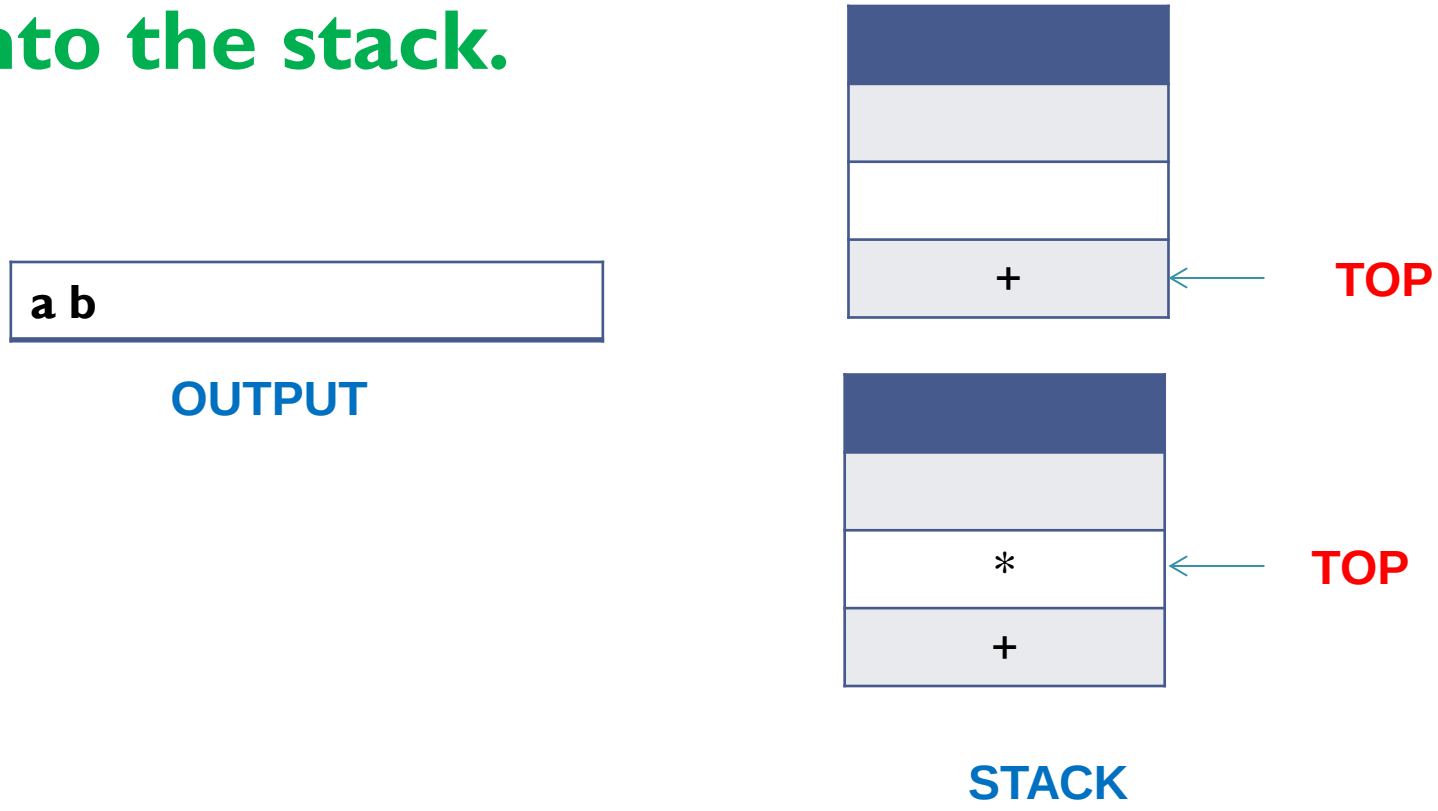
- Next "+" is read, it is pushed into the stack.

- Next "b" is read, it passed through the output.



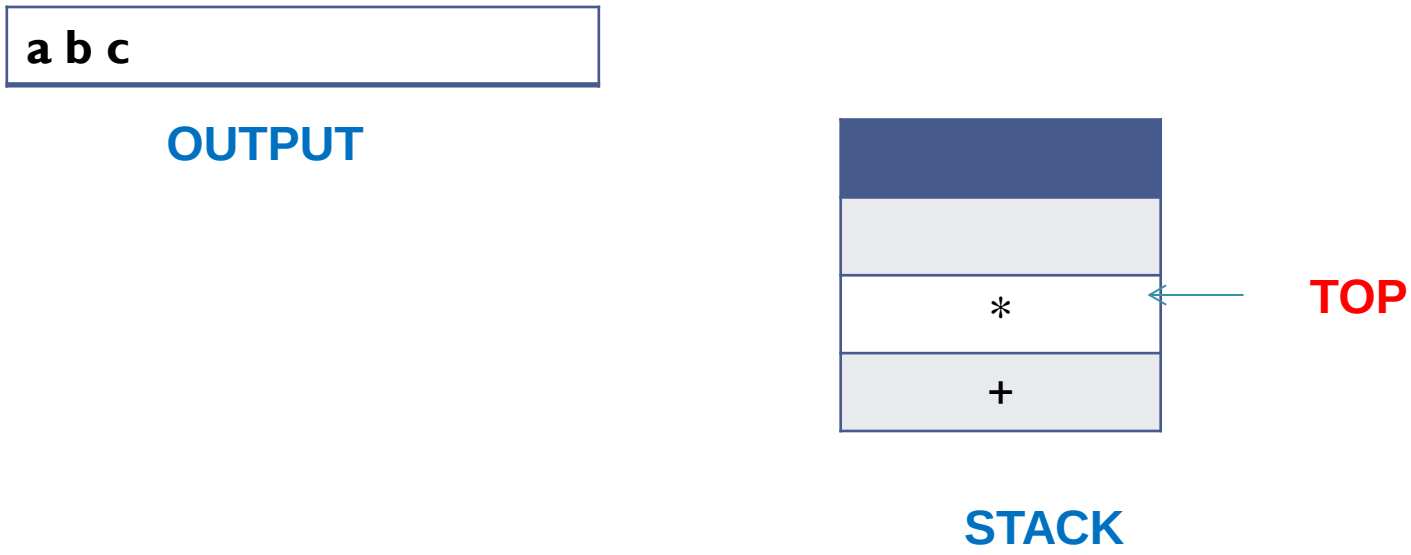
Infix to Postfix Conversion

- Next " * " is read, Check with top entry of the stack. Top entry "+" has lower precedence than *. So push into the stack.



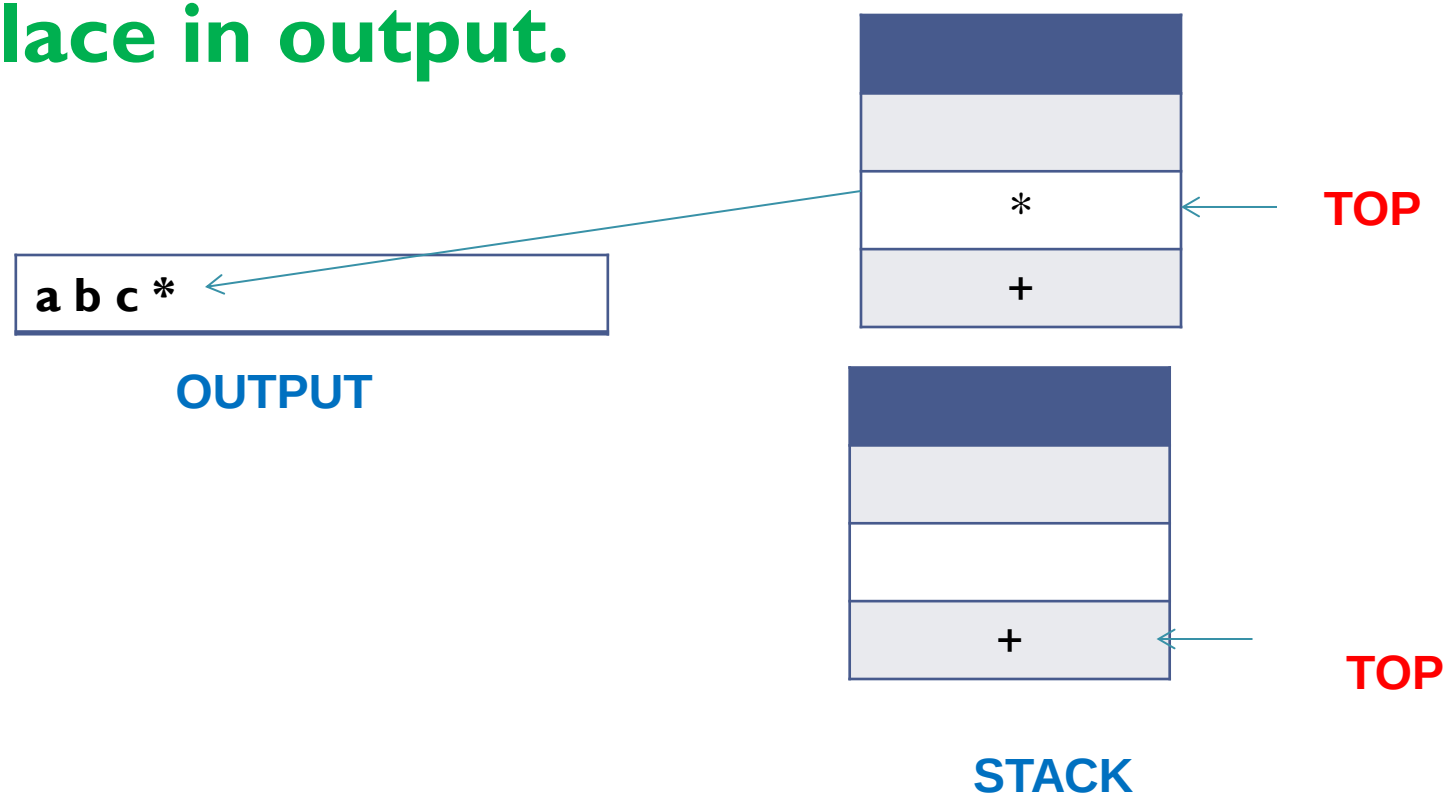
Infix to Postfix Conversion

- Next " c " is read, it is passed to the output.



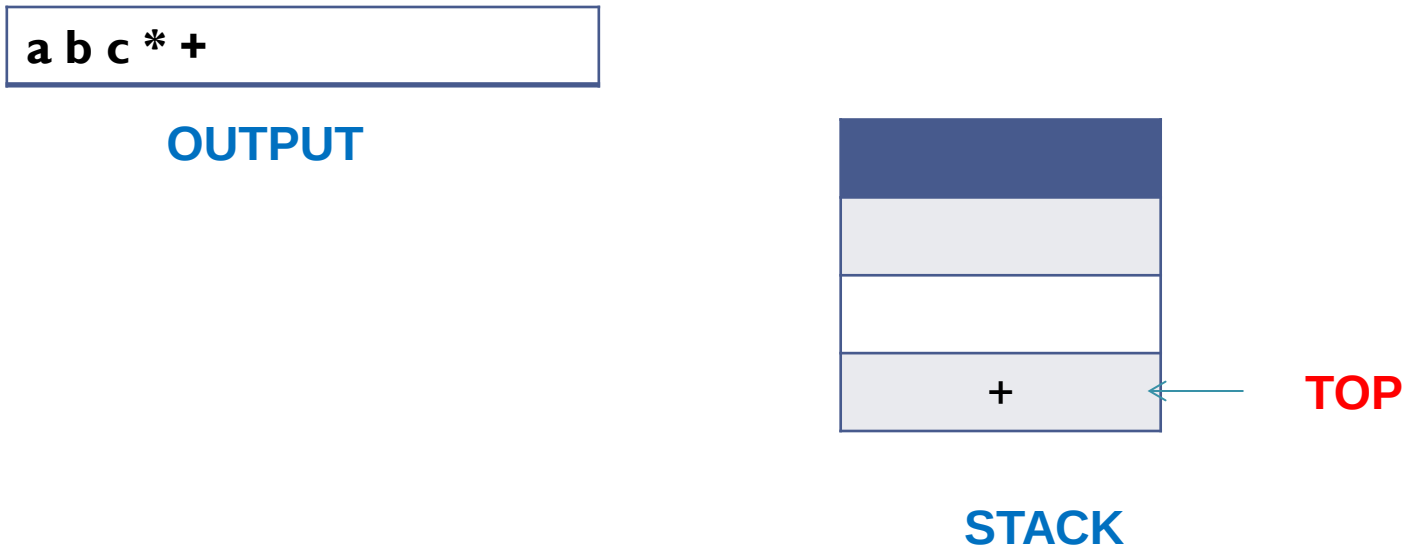
Infix to Postfix Conversion

- Next " + " is read, Check with the top entry of the stack. We have " * "(higher precedence) – so pop it and place in output.



Infix to Postfix Conversion

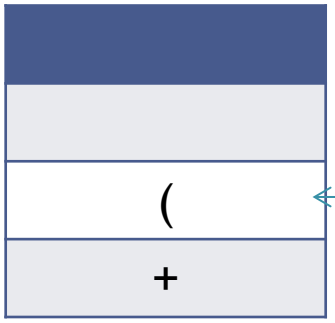
- Next " + " is read, Check with the top entry of the stack. We have " + "(**which is not lower but equal priority) – place into the output**



Infix to Postfix Conversion

- Next " (" is read, Check with the top entry of the stack. We have " + " where "(" is **highest precedence – push into the stack**

OUTPUT

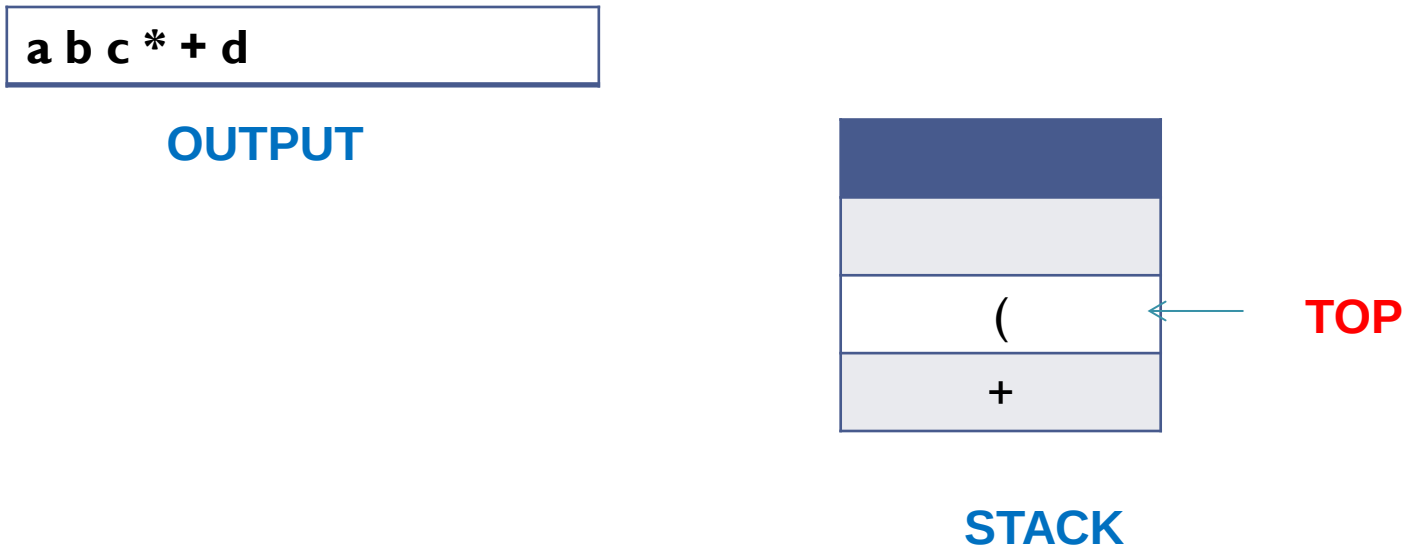


TOP

STACK

Infix to Postfix Conversion

- Next " d " is read, it is passed into the output.

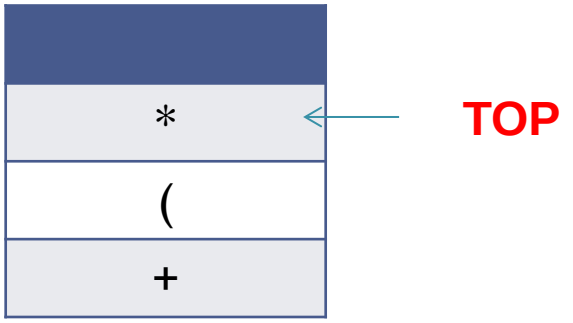


Infix to Postfix Conversion

- Next " * " is read, Check with the top entry of the stack. We have " (" because of **highest precedence it is placed in stack).**
  - **Note: Open parentheses do not get removed except when a closed parenthesis is being processed**

a b c * + d

OUTPUT

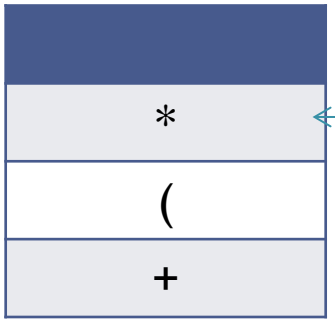


STACK

Infix to Postfix Conversion

- Next " e " is read, it is passed into the output.

OUTPUT

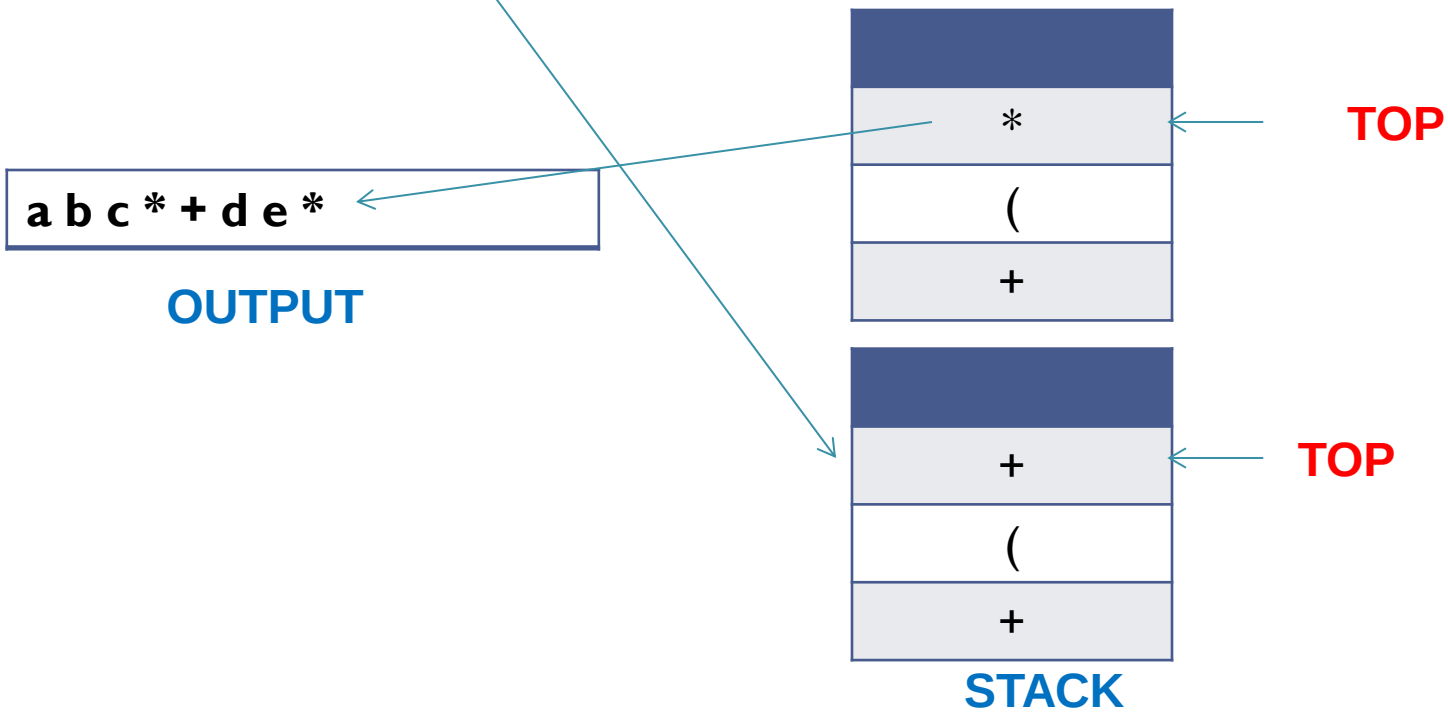


TOP

STACK

Infix to Postfix Conversion

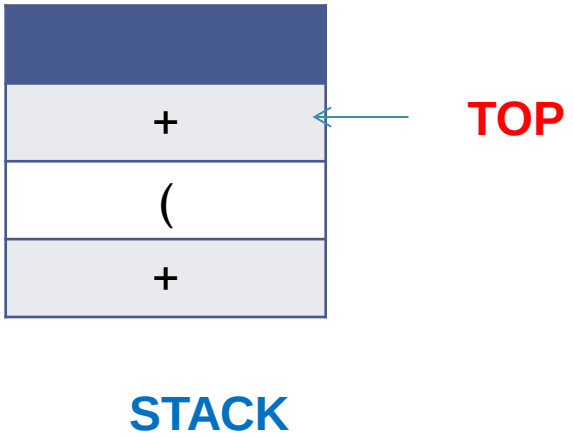
- Next " + " is read, Check with the top entry of the stack. We have " * "(**highest precedence) –** pop out and put into the output. And Push "+" into the stack



Infix to Postfix Conversion

- Next " f " is read, it is passed into the output.

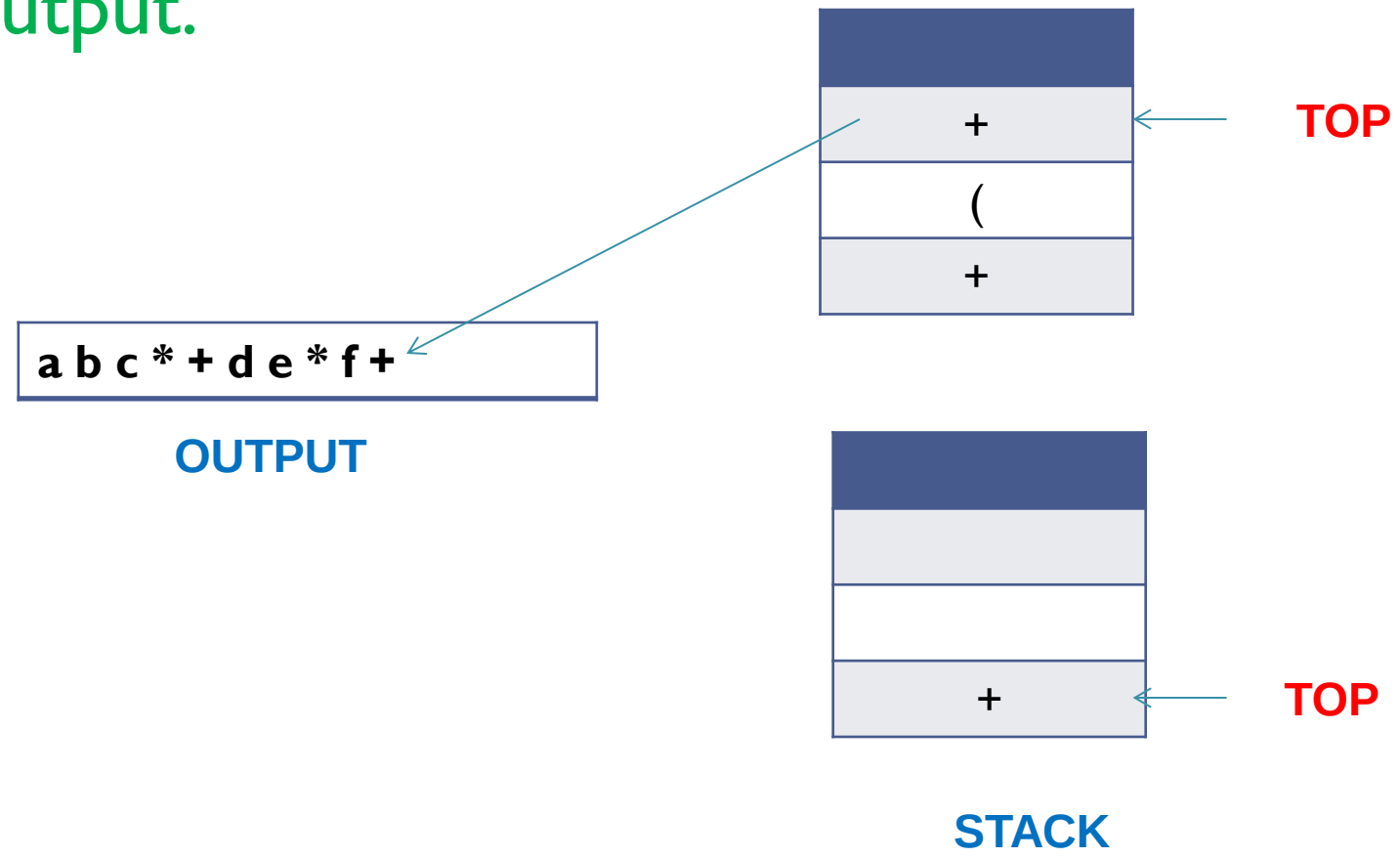
OUTPUT



STACK

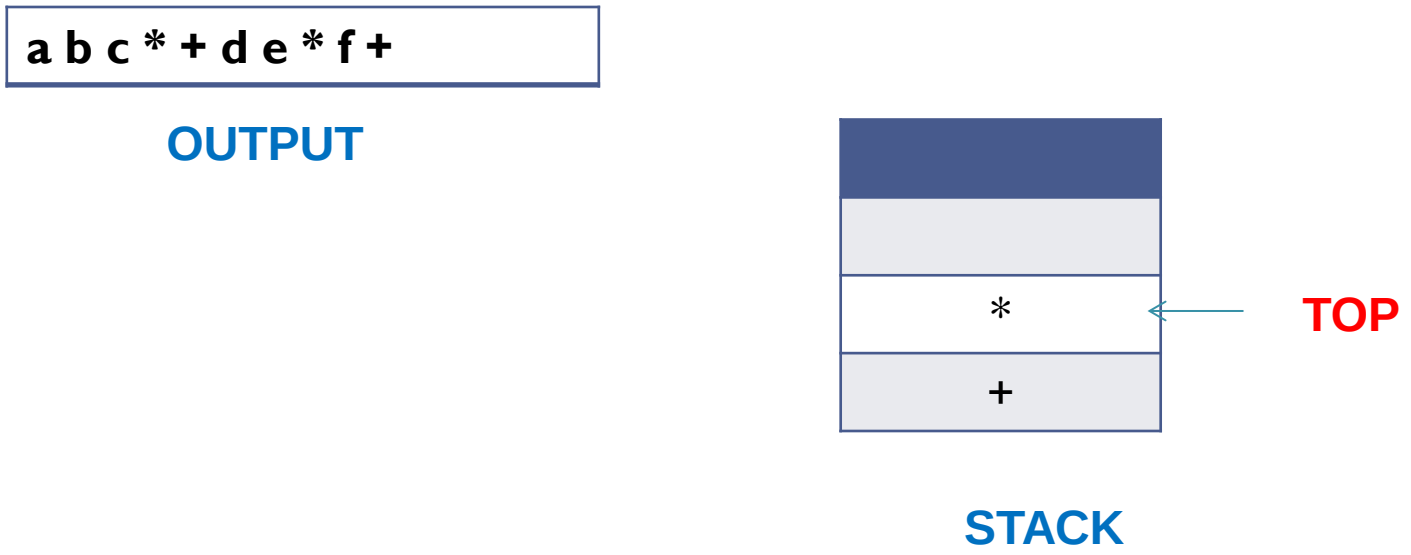
Infix to Postfix Conversion

- Next ") " is read, stack is emptied back upto " (". So " + " is passed into the output.



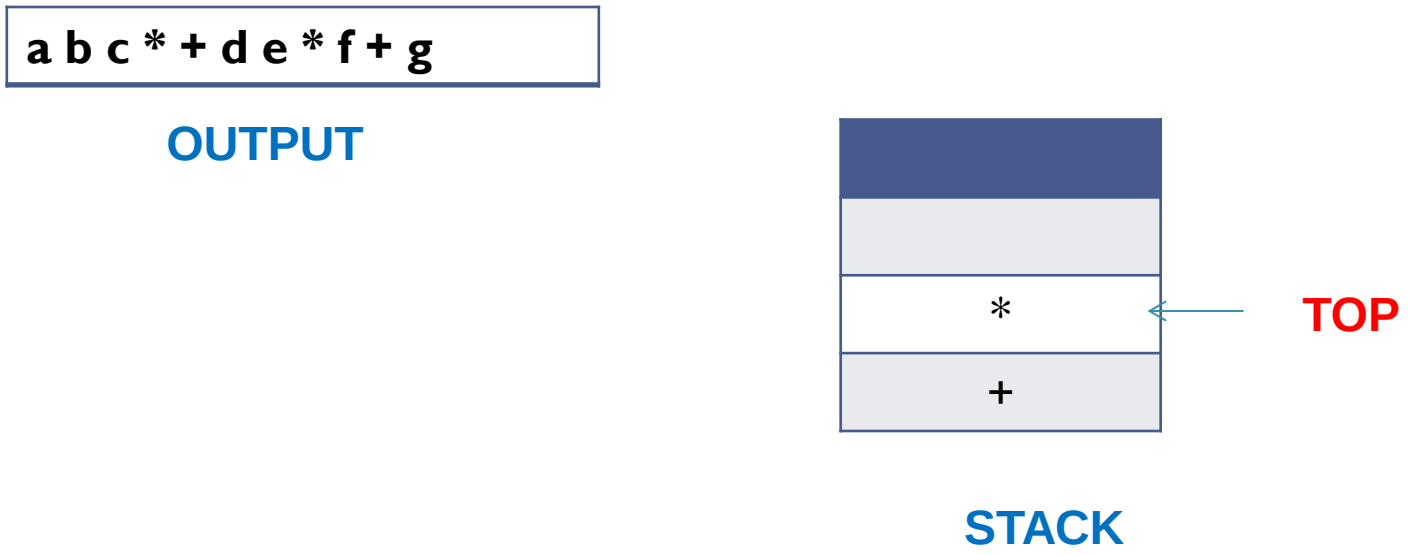
Infix to Postfix Conversion

- Next " * " is read, Check with the top entry of the stack. We have " + "(lower **precedence) – push into the stack**



Infix to Postfix Conversion

- Next " **g** " is read, it is passed into the output.

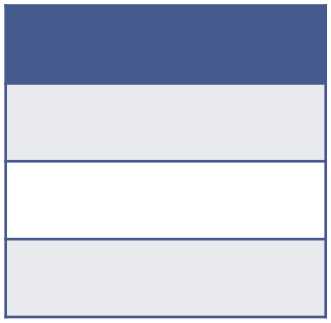


Infix to Postfix Conversion

- Next input is empty , pop all the symbols on the stack until it is empty and move to output.

a b c * + d e * f + g * +

OUTPUT



STACK

TOP

Try Yourself(Infix to Postfix)

- (a-b)/c*(d+e-f/g)
- (4+8)(6-5))/((3-2)(2+2))